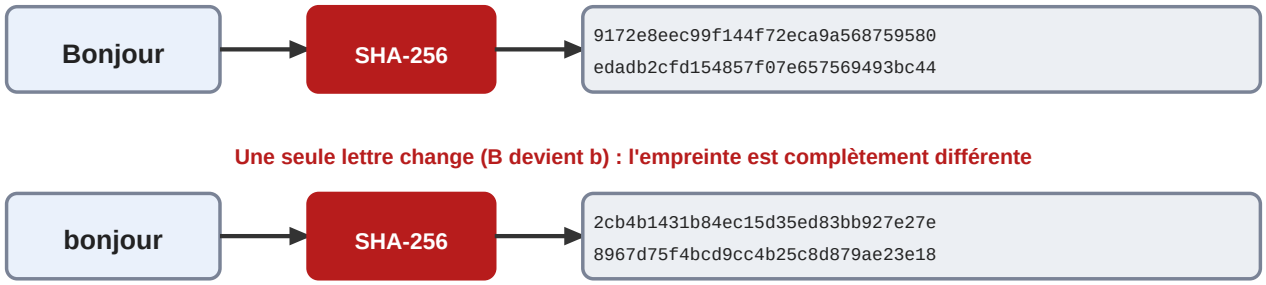
Le hachage

Les empreintes numériques, expliquées simplement

1. C'est quoi le hachage ?

Le **hachage** transforme **n'importe quelle donnée** (un mot, un mot de passe, un fichier de 4 Go...) en une **empreinte** de **taille fixe** appelée **hash** (ou condensat). Cette opération est **à sens unique** : à partir de l'empreinte, on **ne peut pas retrouver** la donnée d'origine.

> **Analogie :** l'**empreinte digitale**. Elle identifie une personne de façon quasi unique, mais on ne peut pas reconstituer la personne à partir de son empreinte.



Empreinte (hash) : toujours 64 caractères hexadécimaux, quelle que soit la taille de l'entrée

2. Les propriétés d'une bonne fonction de hachage

- **Déterministe :** la même donnée donne **toujours** le même hash.
- **Taille fixe :** l'empreinte a toujours la même longueur (64 caractères pour SHA-256).
- **À sens unique :** impossible de remonter à la donnée à partir du hash.
- **Effet avalanche :** changer **un seul caractère** change **tout** le hash.
- **Sans collision :** il doit être (quasi) impossible de trouver deux données différentes avec le même hash.

3. Hachage ou chiffrement ?

	Chiffrement	Hachage
Réversible ?	**Oui**, avec la clé	**Non**, à sens unique
Clé	Oui	Non
Taille du résultat	Proche de celle du message	**Fixe**
But	Garder un message **secret**	**Vérifier** / comparer / contrôler l'intégrité

4. Les algorithmes

Algorithme	Taille du hash	Statut
MD5	32 caractères (128 bits)	**Obsolète** : des collisions ont été trouvées
SHA-1	40 caractères (160 bits)	**Obsolète** : à éviter
SHA-256	64 caractères (256 bits)	**Utilisé** aujourd'hui
bcrypt, Argon2, PBKDF2	variable	Pour les **mots de passe** : volontairement **lents**

Exemples réels (hash du mot `Bonjour`) :

```
MD5     : ebc58ab2cb4848d04ec23d83f7ddf985
SHA-1   : f30ecbf5b1cb85c631fdec0b39678550973cfcbc
SHA-256 : 9172e8eec99f144f72eca9a568759580edadb2cfd154857f07e657569493bc44
```

5. À quoi ça sert ?

• **Stocker les mots de passe :** un site sérieux ne garde **pas** ton mot de passe, mais son **hash**. À la connexion, il hache ce que tu tapes et **compare** les deux empreintes. Si la base est volée, les mots de passe ne sont pas lisibles directement.

• **Vérifier l'intégrité d'un fichier :** après un téléchargement, on compare le hash du fichier avec celui publié par l'éditeur. S'il diffère, le fichier est **corrompu ou modifié**.

• **Signatures numériques et certificats**, **Git**, **blockchain**.

6. Le sel (salt) : protéger les mots de passe

Deux utilisateurs qui choisissent le **même mot de passe** auraient le **même hash**, et des attaquants disposent de tables de hash déjà calculés (**rainbow tables**). Pour éviter cela, on ajoute au mot de passe une valeur aléatoire propre à chaque utilisateur, le **sel**, **avant** de hacher :

```
hash( mot de passe + sel )

motdepasse123 + sel « x9Fq »  ->  un hash
motdepasse123 + sel « k2Lm »  ->  un autre hash (même mot de passe !)
```

7. Calculer un hash soi-même

Système	Commande
Windows (invite de commandes)	`certutil -hashfile fichier.iso SHA256`
Windows (PowerShell)	`Get-FileHash fichier.iso -Algorithm SHA256`
Linux	`sha256sum fichier.iso` ou `md5sum fichier.iso`

8. À retenir

• Le **hachage** donne une **empreinte de taille fixe**, **irréversible**.

• Une modification, même minuscule, change **complètement** l'empreinte.

• On l'utilise pour **vérifier l'intégrité** et **stocker les mots de passe** (avec un **sel**).

• **MD5** et **SHA-1** sont obsolètes : on utilise **SHA-256** ; pour les mots de passe, **bcrypt / Argon2**.